

سامانه ۱۳۷ شهرداری

مستندات پروژه

سرویس ثبت درخواست (Request Service)

۱۲ مرداد ۱۴۰۵

تاریخ نگارش

۱.۰

نسخه مستند

وضعیت پروژه فاز ۱ – «ثبت درخواست» پیاده سازی شده است

ASP.NET Core 8 · EF Core 8 · SQL Server

تکنولوژی

2019

سرویس ثبت درخواست (Request Service) - سامانه ۱۳۷

تاریخ نگارش: ۱۲ مرداد ۱۴۰۵ · نسخه مستند: ۱.۰

وضعیت پروژه: فاز ۱ (فقط «ثبت درخواست» پیاده‌سازی شده است؛ بقیه‌ی مسیرها در انتظار فازهای بعدی هستند)
تکنولوژی: ASP.NET Core 8 · EF Core 8 · SQL Server 2019 · معماری لایه‌ای (Clean Architecture)

۱. معرفی کلی پروژه

این پروژه یک میکروسرویس (سرویس کوچک و مستقل) برای ثبت درخواست شهروندی در سامانه‌ی ۱۳۷ شهرداری است. وقتی شهروندی مشکلی در سطح شهر دارد (مثلاً کولاک، شکستگی جدول، آسفالت یا آب‌گرفتگی)، درخواست او - از طریق اپلیکیشن موبایل/وب شهروندی، اپلیکیشن اپراتور یا مرکز تماس ۱۳۷ - وارد این سرویس می‌شود. سرویس به هر درخواست یک کد پیگیری یکتا می‌دهد (مانند **137-14050521-000004**) که شهروند به کمک آن می‌تواند وضعیت درخواستش را بعداً پیگیری کند.

کارکرد اصلی این سرویس «دریافت درخواست و نگهداری امن آن» است؛ یعنی:

- هویت فرستنده‌ی درخواست را بررسی می‌کند (آیا واقعاً یک شهروند لاگین‌شده است، یا اپراتور، یا سامانه‌ی تلفنی).
- مشخصات درخواست را ذخیره می‌کند (توضیح، موقعیت مکانی، فایل‌های ضمیمه).
- یک بایگانی کامل از همه‌ی رویدادهای هر درخواست نگه می‌دارد (چه کسی، چه زمانی، چه کاری با درخواست انجام داد) تا قابلیت پیگیری و پاسخگویی کامل باشد.

فایل‌های ضمیمه و هویت شهروند در خود این سرویس ذخیره نمی‌شوند؛ این سرویس به دو سرویس دیگر (سرویس فایل و سرویس ورود یکتا / SSO) تکیه می‌کند که در بخش ۶ توضیح داده شده‌اند.

۲. معماری کلی

این پروژه یک میکروسرویس مستقل در خانواده‌ی سرویس‌های سامانه‌ی ۱۳۷ است و با معماری لایه‌ای (Clean Architecture) نوشته شده است. کد در ۴ لایه از هم جدا شده‌اند:

src/

- RequestService.Domain → مدل‌های اصلی و مفاهیم دامنه (درخواست، فایل، لاگ، وضعیت‌ها)
- RequestService.Application → منطق کسب‌وکار: اعتبارسنجی، احراز هویت، هماهنگ‌سازی (ارکستراسیون)
- RequestService.Infrastructure → فراخوانی سرویس‌های دیگر (EF Core)، اتصال به دیتابیس
- RequestService.Api → تنظیمات برنامه، Swagger، میان‌افزار خطاهای Controller: لایه‌ی نمایش

نمودار ارتباطی سرویس (Mermaid):

نمودار ارتباطی سرویس



نکته‌ی معماری مهم: سرویس بدون حال (stateless) است - حافظه یا کش درونی ندارد؛ تمامی داده در SQL Server نگهداری می‌شود و هویت/فایل‌ها به سرویس‌های دیگر ارجاع داده می‌شوند، نه این‌که این‌جا کپی شوند.

۳. نقش‌های کاربری

نقش	چه کسی	نحوه‌ی ورود به سرویس	سطح دسترسی فعلی
Citizen (شهروند)	شهروند از اپ موبایل/وب شهروندی	توکن ورود یکتا (Bearer)	فقط ثبت درخواست
Operator (اپراتور)	کارشناس شهرداری از اپ اپراتور یا مرکز تماس	توکن ورود یکتا (Bearer)	ثبت درخواست (بعداً کارتابل/مدیریت)
ExternalService (سرویس تلفنی / IVR)	سامانه‌ی تلفن مرکز ۱۳۷	کلید API (X-API-Key)	ثبت درخواست تلفنی
InternalService (سرویس داخلی)	سایر سرویس‌های داخلی شهرداری	کلید API (X-API-Key) - اجباری	ثبت درخواست به‌نماینده‌ی
System (سیستم)	اقدامات خودکار سیستم	-	فقط در لاگ رویدادها ثبت می‌شود

«ادمین» یا نقش‌های مدیریتی هنوز تعریف نشده‌اند؛ مدیریت کارتابل (تغییر وضعیت، ارجاع، جستجو) در فازهای بعدی می‌آید (بخش ۴.۹).

۴. فهرست کامل بخش‌ها / قابلیت‌ها

۴.۱ ثبت درخواست جدید (`POST /api/v1/requests`)

این بخش چه کاری انجام می‌دهد؟

درخواست شهروند را از هر کانال (تلفن، اپ شهروندی، اپ اپراتور، سرویس داخلی) دریافت می‌کند، صحت و هویت فرستنده را می‌سنجد، فایل‌های ضمیمه را (در صورت فعال بودن) در سرویس فایل تأیید می‌کند، یک کد پیگیری یکتا تولید می‌کند و همه‌چیز را در یک تراکنش واحد ذخیره می‌کند. این تنها قابلیت کاملاً پیاده‌سازی‌شده‌ی سرویس در فاز ۱ است.

چه کسی از آن استفاده می‌کند؟

هر چهار نقش ورودی (شهروند، اپراتور، سرویس تلفنی، سرویس داخلی) - بسته به کانال ارسال درخواست.

جزئیات عملکرد (مرحله به مرحله):

۱. اعتبارسنجی ورودی - بدنه‌ی درخواست با قوانین وابسته به کانال بررسی می‌شود (بخش ۴.۳). اگر خطا داشته باشد، خطای ۴۰۰ با کد `VALIDATION_ERROR` برگردانده می‌شود.
۲. شناسایی فرستنده - بسته به کانال، کلید API یا توکن ورود یکتا بررسی و «هویت تماس‌گیرنده» مشخص می‌شود (بخش ۴.۲).
۳. تعیین کد ملی - برای کانال‌های شهروندی، کد ملی از توکن گرفته می‌شود (توکن معتبر است)؛ برای تلفن/سرویس داخلی می‌تواند از بدنه بیاید یا خالی بماند.
۴. تأیید فایل‌ها - اگر `fileds` فرستاده شده باشند، هر کدام با سرویس فایل چک می‌شوند (بخش ۴.۶) و نوع فایل (صوتی/تصویری/ویدیویی/سایر) تعیین می‌شود.
۵. تولید کد پیگیری - از دنباله‌ی دیتابیس یک شماره‌ی یکتا گرفته و با تاریخ شمسی ترکیب می‌شود (بخش ۴.۴).
۶. ساخت موجودیت‌ها - درخواست، ردیف‌های فایل ضمیمه و رویداد اولیه‌ی «ساخته شد» ساخته می‌شوند.
۷. ذخیره‌سازی اتمیک - همه با هم در یک تراکنش دیتابیس ثبت می‌شوند (بخش ۴.۷). اگر وسط کار خطایی رخ دهد، هیچ‌چیز ذخیره نمی‌شود.
۸. پاسخ - شناسه‌ی درخواست و کد پیگیری برمی‌گردد (HTTP 201).

ورودی / خروجی مهم:

• مسیر و متد: `POST /api/v1/requests` - هدرهای موردنیاز: `Content-Type: application/json` + کلید API یا توکن (بخش ۴.۲)

• بدنه‌ی ورودی (همه فیلدها اختیاری‌اند مگر الزام کانال؛ برای برخی کانال‌ها قوانین بیشتری هست - بخش ۴.۳):

فیلد ورودی	نوع	توضیح ساده
channel	رشته	کانال ورود درخواست: CitizenMobileApp ، PhoneCall ، InternalService ، OperatorApp ، CitizenWebApp
citizen.nationalCode	رشته	کد ملی ۱۰ رقمی شهروند
citizen.firstName / citizen.lastName	رشته	نام و نام خانوادگی (فقط برای کانال تلفنی که هنوز شهروند در SSO ثبت نشده)
citizen.phoneNumber	رشته	شماره تلفن
description	رشته	شرح مشکل (حداکثر ۲۰۰۰ نویسه)
location.lng / location.lat	عدد	مختصات مکانی درخواست (برای اپ شهروندی اجباری است)
fields	فهرست رشته‌ای	شناسه‌های فایل در سرویس فایل (حداکثر ۲۰ فایل، بدون تکرار)

• خروجی (HTTP 201): { "requestId": "<شناسه‌ی یکتای درخواست>", "trackingCode": "137-14050521-000004" }

• خطاها: ۴۰۰/۴۰۱/۵۰۳ با قالب یکنواخت (بخش ۴.۸)

جدول‌های دیتابیس مرتبط با این بخش (مالک اصلی: این بخش):

جدول: Requests

توضیح ساده: نگهدارنده‌ی «وضعیت فعلی» هر درخواست - مشخصات اصلی درخواست همان‌طور که الان هست.

نام ستون	نوع داده	اجباری؟	توضیح ساده
Id	uniqueidentifier (Guid)	بله	شناسه یکتای درخواست
TrackingCode	nvarchar(32)	بله	کد پیگیری نمایشی شهروند (یکتا)
NationalCode	nvarchar(20)	خیر	کد ملی شهروند (در حالت تلفنی بدون هویت تأییدشده می‌تواند خالی باشد)
Description	nvarchar(2000)	خیر	شرح درخواست
LocationLat	decimal(18,6)	خیر	عرض جغرافیایی
LocationLng	decimal(18,6)	خیر	طول جغرافیایی
Channel	nvarchar(32)	بله	کانال ورود (به‌صورت متن ذخیره می‌شود)
Status	nvarchar(32)	بله	وضعیت فعلی (به‌صورت متن)

نام ستون	نوع داده	اجباری؟	توضیح ساده
CurrentGroupId	nvarchar(64)	خیر	شناسه‌ی واحد/گروه مسئول فعلی (تا زمان ارجاع خالی است)
CreatedBySourcePhone	nvarchar(20)	خیر	شماره‌ی تلفن منبع در سناریوی مرکز تماس
CreatedAtUtc	datetime2	بله	زمان ثبت درخواست (به UTC)
UpdatedAtUtc	datetime2	خیر	زمان آخرین تغییر (فعلاً در فاز ۱ استفاده نمی‌شود)

ارتباطات (Relations):

- Requests.Id ← RequestFiles.RequestId (هر درخواست می‌تواند چند فایل ضمیمه داشته باشد)
- Requests.Id ← RequestLogs.RequestId (هر درخواست چندین رویداد بایگانی دارد)

ایندکس‌ها / محدودیت‌های مهم:

- TrackingCode یکتا (Unique) - تضمین عدم تکرار کد پیگیری
- ایندکس روی NationalCode ، CurrentGroupId ، CreatedAtUtc برای جستجوی آینده

جدول: RequestFiles

توضیح ساده: پیوند بین درخواست و فایل ضمیمه - فقط شناسه‌ی فایل (صادر شده از سرویس فایل) ذخیره می‌شود، محتوای فایل هرگز این‌جا ذخیره نمی‌شود.

نام ستون	نوع داده	اجباری؟	توضیح ساده
Id	uniqueidentifier (Guid)	بله	شناسه بکتای ردیف پیوند
RequestId	uniqueidentifier (Guid)	بله	به کدام درخواست تعلق دارد
FileId	nvarchar(64)	بله	شناسه‌ی فایل در سرویس فایل
FileType	nvarchar(16)	بله	نوع فایل: Other ، Video ، Image ، Audio
CreatedAtUtc	datetime2	بله	زمان افزودن

ارتباطات (Relations):

- RequestFiles.RequestId → Requests.Id (هر درخواست چند فایل می‌تواند داشته باشد؛ حذف درخواست = حذف خودکار این ردیف‌ها - Cascade)

ایندکس‌ها / محدودیت‌های مهم:

- ایندکس یکتا روی (RequestId, FileId) - جلوگیری از الحاق دوباره‌ی یک فایل به یک درخواست

جدول: RequestLogs

توضیح ساده: بایگانی «فقط‌افزودنی» از همه‌ی رویدادهای هر درخواست (چه کسی، چه زمانی، چه عملی انجام داد). ردیف‌های این جدول هیچ‌وقت ویرایش یا حذف نمی‌شوند.

نام ستون	نوع داده	اجباری؟	توضیح ساده
Id	uniqueidentifier (Guid)	بله	شناسه یکتای رویداد
RequestId	uniqueidentifier (Guid)	بله	مربوط به کدام درخواست
ActionType	nvarchar(32)	بله	نوع عمل: Created ، ReviewedByOperator ، Confirmed ، Referred ، Rejected ، StatusChanged ، Closed و ...
ActorType	nvarchar(32)	بله	چه کسی انجام داد: Citizen ، Operator ، System ، ExternalService
ActorId	nvarchar(64)	خیر	شناسه‌ی انجام‌دهنده (کد پرسنلی اپراتور / کد ملی شهروند)
PreviousStatus / NewStatus	nvarchar(32)	خیر	وضعیت قبل/بعد (برای تغییر وضعیت‌ها)
PreviousGroupId / NewGroupId	nvarchar(64)	خیر	گروه قبل/بعد (برای ارجاع‌ها)
Description	nvarchar(4000)	خیر	جزئیات آزاد (در فاز ۱، نام/تلفن موقت تلفنی به‌صورت JSON این‌جا نگهداری می‌شود)
CreatedAtUtc	datetime2	بله	زمان رویداد

ارتباطات (Relations):

- **RequestLogs.RequestId** → **Requests.Id** (هر درخواست چند رویداد بایگانی دارد؛ حذف درخواست = حذف خودکار رویدادها - Cascade)

ایندکس‌ها / محدودیت‌های مهم:

- ایندکس روی (**RequestId, CreatedAtUtc**) - نمایش تاریخچه‌ی یک درخواست به ترتیب زمان

در فاز ۱ فقط رویداد **Created** با **ActorType** برابر نوع فرستنده ثبت می‌شود. سایر نوع‌های **ActionType** برای فازهای بعدی تعریف شده‌اند.

وابستگی‌ها:

- سرویس ورود یکتا (SSO) برای اعتبارسنجی توکن (بخش ۴.۵)

- سرویس فایل برای تأیید شناسه‌ی فایل‌ها (بخش ۴.۶)

- SQL Server برای ذخیره‌سازی (بخش ۴.۷)

نکات و استثناها:

- **تکرار نشدن PII:** طبق تصمیم معماری، نام/نام خانوادگی/تلفن دریافت‌شده در تماس تلفنی در جدول **Requests** ذخیره نمی‌شود؛ فقط به‌صورت JSON داخل **RequestLogs.Description** (بایگانی) ثبت می‌شود تا هویت شهروند در سرویس SSO تکمیل شود. این مصالحه در بخش ۷ «نیاز به بررسی» ثبت شده است.
- حداکثر ۲۰ فایل و بدون تکرار در **filelds** مجاز است.
- اگر **Files:ValidationEnabled=false** باشد (در حالت توسعه)، وجود فایل‌ها چک نمی‌شود و نوع فایل همیشه **Other** ذخیره می‌شود.

۴.۲ احراز هویت و شناسایی فرستنده

این بخش چه کاری انجام می‌دهد؟

قبل از ذخیره‌ی هر درخواست مشخص می‌کند «چه‌کسی» درخواست را می‌دهد و بر اساس کانال، قانون‌های ورود را اجرا می‌کند.

چه کسی از آن استفاده می‌کند؟

همه‌ی فرستنده‌های درخواست - مکانیزم شفاف است و فرستنده‌ی معمولی چیزی از آن نمی‌بیند.

جزئیات عملکرد:

1. هدر **Authorization: Bearer ...** (توکن ورود یکتا) و هدر **X-API-Key** (کلید داخلی) از درخواست خوانده می‌شوند.
 2. بر اساس کانال:
 - **InternalService** → کلید **API الزامی** است؛ هویت «سرویس داخلی» با نام کلید ثبت می‌شود.
 - **PhoneCall** → کلید API کافی است (سامانه‌ی تلفنی) یا اگر توکن اپراتور باشد، اپراتور می‌شود.
 - **OperatorApp** / **CitizenMobileApp** / **CitizenWebApp** → توکن ورود یکتا **الزامی** است و در سرویس SSO اعتبارسنجی می‌شود. 3. اگر کلید/توکن معتبر نباشد → خطای ۴۰۱ **UNAUTHORIZED**.
 4. کلیدهای مجاز در تنظیمات **InternalAuth:ApiKeys** تعریف شده‌اند (در حالت توسعه: **dev-internal-key-137**).
- ورودی / خروجی مهم:** هدرها **X-API-Key** و **Authorization: Bearer <token>** - خروجی مستقیم ندارد؛ نتیجه در بایگانی ثبت می‌شود.

جدول‌های دیتابیس مرتبط: این بخش جدول مستقلی ندارد؛ نتیجه‌ی آن در **RequestLogs.ActorType** / **ActorId** (که در بخش ۴.۱ توضیح داده شد) ثبت می‌شود.

وابستگی‌ها: سرویس SSO برای اعتبارسنجی توکن؛ فایل تنظیمات **InternalAuth** برای کلیدهای داخلی.

نکات و استثناها:

- کلید API فقط برای کانال‌های «تلفنی/سرویس داخلی» پذیرفته می‌شود؛ کانال‌های شهروندی فقط توکن می‌پذیرند.
- **ناسازگاری ثبت‌شده در کد:** در کد بخش ۴.۱ این جمله آمده که «در کانال‌های شهروندی، کد ملی ارسالی در بدنه اگر مغایر

با توکن باشد رد می‌شود» ولی در عمل چنین قانون اعتبارسنجی‌ای پیاده‌سازی نشده و فقط «کد ملی توکن» ملاک است. این مورد در بخش ۷ علامت خورده است.

۴.۳ اعتبارسنجی ورودی (FluentValidation)

این بخش چه کاری انجام می‌دهد؟

قبل از هر پردازشی، بدنه‌ی درخواست را با قوانین وابسته به کانال بررسی می‌کند تا داده‌ی ناقص/نادرست وارد سیستم نشود.

چه کسی از آن استفاده می‌کند؟ همه‌ی فرستنده‌ها - به صورت خودکار.

جزئیات عملکرد: قوانین به ترتیب در کلاس `CreateRequestValidator` اجرا می‌شوند؛ اولین پیام خطا همراه با `VALIDATION_ERROR` (۴۰۰) برمی‌گردد.

قوانین:

- `channel` باید یکی از ۵ مقدار مجاز باشد.
- کانال‌های `CitizenWebApp` / `CitizenMobileApp`: فیلد `location` اجباری است.
- `location.lat` بین ۹۰- تا ۹۰ و `location.lng` بین ۱۸۰- تا ۱۸۰ (اگر ارسال شده باشد).
- کانال `OperatorApp`: `citizen.nationalCode` اجباری است.
- `nationalCode` (اگر ارسال شده باشد): دقیقاً ۱۰ رقم.
- `description` حداکثر ۲۰۰۰ نویسه؛ `lastName` / `firstName` حداکثر ۱۰۰ نویسه.
- `phoneNumber` (اگر ارسال شده باشد): قالب شماره‌ی تلفن (۷ تا ۲۰ نویسه، شامل ارقام و `- +` و فاصله).
- `fileIds`: حداکثر ۲۰ مورد، هرکدام باید شناسه‌ی معتبر GUID باشد، بدون تکرار.

ورودی / خروجی مهم: ورودی: همان بدنه‌ی بخش ۴.۱ خروجی در حالت خطا: ۴۰۰ با `{code: "VALIDATION_ERROR", message, traceId}`

جدول‌های دیتابیس مرتبط: هیچ - اعتبارسنجی قبل از دیتابیس انجام می‌شود.

وابستگی‌ها: کتابخانه‌ی `FluentValidation` (وابستگی داخلی NuGet).

نکات و استثناها: در کد این کلاس یک متغیر `VideoExtensions` تعریف شده که استفاده نمی‌شود (کد مرده) - در بخش ۷ علامت خورده است.

۴.۴ تولید کد پیگیری یکتا

این بخش چه کاری انجام می‌دهد؟

برای هر درخواست یک کد پیگیری خوانا و یکتا می‌سازد تا شهروند بتواند درخواستش را پیگیری کند.

چه کسی از آن استفاده می‌کند؟ شهروند و اپراتور (به عنوان شماره‌ی پیگیری) و سرویس به صورت خودکار.

جزئیات عملکرد:

- از دنباله‌ی دیتابیس `dbo.RequestTrackingCodeSeq` یک شماره‌ی ترتیبی و یکتا گرفته می‌شود (با دستور مستقیم `NEXT VALUE FOR`).
- تاریخ شمسی (تقویم جلالی) از ساعت سیستم به شکل `yyyyMMdd` گرفته می‌شود.
- کد به این قالب ساخته می‌شود: `<تاریخ شمسی>-<شماره‌ی ۶رقمی>-137` مثل `137-14050521-000004`.
- برای اطمینان، وجود کد در جدول `Requests` یک‌بار دیگر چک می‌شود (در صورت تکرار، یک شماره‌ی جدید ساخته می‌شود - احتمال وقوع بسیار کم است).

ورودی / خروجی مهم: خروجی: رشته‌ی کد پیگیری (۲۱ نویسه). هیچ ورودی از کاربر ندارد.

جدول‌های دیتابیس مرتبط: از ستون `Requests.TrackingCode` (که در بخش ۴.۱ توضیح داده شد) و از دنباله‌ی `dbo.RequestTrackingCodeSeq` استفاده می‌کند. تعریف دنباله در migration: از ۱ شروع می‌شود، گام ۱، بدون چرخش (NO CYCLE)، با حافظه‌ی کش ۵۰.

وابستگی‌ها: SQL Server (دنباله + ایندکس یکتا).

نکات و استثناها:

- امنیت همزمانی (Race): چون شماره از دنباله‌ی دیتابیس گرفته می‌شود و ستون `TrackingCode` ایندکس یکتا دارد، حتی با ارسال هم‌زمان چند درخواست هم کدها تکراری نمی‌شوند. این مورد با تست هم‌زمانی (۵ درخواست موازی) تأیید شده است.
- تاریخ کد شمسی بر اساس ساعت سرور محاسبه می‌شود (نه UTC).

۴.۵ یکپارچه‌سازی با سرویس ورود یکتا (SSO)**این بخش چه کاری انجام می‌دهد؟**

توکن ورود یکتا (JWT) فرستنده‌ها را به سرویس SSO می‌فرستد تا اعتبار آن تأیید و هویت شهروند (کد ملی و تلفن) استخراج شود. اگر سرویس SSO در دسترس نباشد، خطای شفاف ۵۰۳ برمی‌گردد (نه عبور بی‌هویت، نه کرش).
چه کسی از آن استفاده می‌کند؟ شهروندها و اپراتورها که با توکن وارد می‌شوند.

جزئیات عملکرد:

- توکن از هدر `Authorization` گرفته می‌شود.
- `GET <Sso:BaseUrl>/api/auth/me` با هدر `Authorization: Bearer <token>` فراخوانی می‌شود.
- پاسخ ۴۰۱/۴۰۳ → توکن نامعتبر (نتیجه `null` → خطای ۴۰۱ در سرویس).
- پاسخ موفق → کد ملی و تلفن از پاسخ استخراج می‌شود.
- خرابی شبکه/وقفه → خطای ۵۰۳ `DEPENDENCY_UNAVAILABLE`.
- مقاوم‌سازی (Resilience): تلاش مجدد + مدارشکن (Circuit Breaker) با تنظیمات `Sso:*` فعال است.

ورودی / خروجی مهم: ورودی: توکن · خروجی: شناسه‌ی کاربر، کد ملی، تلفن (یا نامعتبر/خطا).

جدول‌های دیتابیس مرتبط: ندارد.

وابستگی‌ها: سرویس `sso-login-service` (پورت ۵۰۰۱ در توسعه، آدرس از `Sso:BaseUrl`).

نکات و استثناها:

- قالب پاسخ SSO مطابق سرویس واقعی `sso-login-service` (فیلدهای `success`، `data`، `id`، `melliCode`، `phone`) ساخته شده است.
- بلاکر (نیاز به بررسی): از این کامپیوتر توسعه، سرویس SSO در دسترس نیست؛ تست واقعی توکن باید روی سرور انجام شود.

۴.۶ یکپارچه‌سازی با سرویس فایل

این بخش چه کاری انجام می‌دهد؟

برای هر `fileId` فرستاده‌شده بررسی می‌کند که آن فایل واقعاً در سرویس فایل وجود دارد و نوعش (صوتی/تصویری/ویدیویی/سایر) را تشخیص می‌دهد. محتوای فایل هرگز دانلود یا در این سرویس ذخیره نمی‌شود - فقط متادیتا (شناسه، پسوند، نوع MIME) چک می‌شود.

چه کسی از آن استفاده می‌کند؟ همه‌ی کانال‌ها هنگام ارسال `fileId` (مثلاً ضبط صوتی تماس یا عکس از اپ).

جزئیات عملکرد:

- مسیر `GET <Files:BaseUrl>/api/files/{id}` با جای‌گذاری شناسه‌ی فایل ساخته می‌شود.
 - اگر `Files:ServiceToken` تنظیم شده باشد، به صورت `Bearer` ارسال می‌شود (احراز هویت سرویس به سرویس).
 - پاسخ ۴۰۴ → فایل وجود ندارد (خطای اعتبارسنجی).
 - پاسخ ۴۰۱/۴۰۳ → توکن سرویس رد شده (خطای ۵۰۳ با پیام راهنما).
 - پاسخ موفق → پسوند/نوع MIME خوانده شده و نوع فایل (`Other` / `Video` / `Image` / `Audio`) با تطبیق پسوند/نوع تشخیص داده می‌شود.
 - اگر `Files:ValidationEnabled=false` باشد، کل این مرحله رد می‌شود و نوع همیشه `Other` است (حالت توسعه).
- ورودی / خروجی مهم: ورودی: `fileId` · خروجی: نوع فایل یا خطای اعتبارسنجی/وابستگی.
- جدول‌های دیتابیس مرتبط: نتیجه در `RequestFiles.FileType` ذخیره می‌شود (جدول در بخش ۴.۱ توضیح داده شد).
- وابستگی‌ها: سرویس `files` (پورت ۶۰۰۰ در توسعه، آدرس از `Files:BaseUrl`).

نکات و استثناها:

- بلاکر (نیاز به بررسی): توکن سرویس (`Files:ServiceToken`) هنوز از طرف تیم سرویس فایل صادر نشده است. تا آن موقع، اعتبارسنجی واقعی فایل فعال نیست (`ValidationEnabled=false` در توسعه). این مورد در بخش ۷ ثبت شده است.

۴.۷ ذخیره‌سازی تراکنشی در دیتابیس

این بخش چه کاری انجام می‌دهد؟

همه‌ی عملیات نوشتن (درخواست + فایل‌ها + رویداد بایگانی) را در یک تراکنش واحد انجام می‌دهد تا یا همه ذخیره شوند یا هیچ‌کدام.

چه کسی از آن استفاده می‌کند؟ به صورت داخلی هنگام ثبت درخواست.

جزئیات عملکرد:

1. با استفاده از استراتژی اجرای تراکنشی EF Core (`CreateExecutionStrategy`) یک تراکنش باز می‌شود - این روش لازم است چون EF برای خطاهای گذرا (مثل قطع کوتاه شبکه) کل واحد کار را دوباره اجرا می‌کند.
2. در هر اجرا، حالت ردیابی (`Change Tracker`) پاک می‌شود تا اگر تراکنش دوباره اجرا شد، ردیف‌ها دوباره‌ای ثبت نشوند (درج تکرارنشونده).
3. ردیف‌های `Requests`، `RequestFiles` و `RequestLogs` اضافه و با هم ثبت می‌شوند؛ سپس تراکنش `commit` می‌شود.
4. اگر خطایی رخ دهد، تراکنش برگشت می‌خورد (`Rollback`) و هیچ داده‌ای نمی‌ماند.
5. شماره‌ی دنباله با دستور مستقیم `SELECT NEXT VALUE FOR dbo.RequestTrackingCodeSeq` خوانده می‌شود (EF اجازه‌ی استفاده در جستارهای معمولی را نمی‌دهد).

ورودی / خروجی مهم: داخلی - API عمومی ندارد.

جدول‌های دیتابیس مرتبط: هر سه جدول (بخش ۴.۱) و دنباله (بخش ۴.۴).

وابستگی‌ها: SQL Server 2019 (رشته اتصال از `ConnectionStrings:Requests`).

نکات و استثناها:

- در طول توسعه، یک خطای ۵۰۰ (نقض کلید خارجی در درج هم‌زمان `RequestFiles`) پیش آمد که با اتصال صریح رابطه‌ی `file.Request = entity` برطرف شد. این باگ حل شده و تست شده است.
- روش‌های `UpdateStatusAsync`، `ReferAsync`، `SearchAsync`، `GetByIdAsync` و `GetByTrackingCodeAsync` در این لایه نوشته شده‌اند اما هنوز به هیچ API متصل نشده‌اند (مربوط به فاز ۲ - بخش ۷).

۴.۸ قالب یکنواخت خطاها

این بخش چه کاری انجام می‌دهد؟

همه‌ی خطاها (اعم از خطای ورودی، احراز هویت، وابستگی و خطای غیرمنتظره) را به یک ساختار JSON یکسان تبدیل می‌کند تا مصرف‌کننده‌های API همیشه یک قرارداد ثابت ببینند.

چه کسی از آن استفاده می‌کند؟ همه‌ی مصرف‌کننده‌های API.

جزئیات عملکرد: میان‌افزار `ExceptionHandlerMiddleware` هر استثنای پرتاب‌شده را گرفته و به پاسخ HTTP نگاشت می‌کند. `traceld` برای ردیابی خطا در لاگ‌ها استفاده می‌شود.

قالب: `{ "code": "...", "message": "...", "traceld": "..." }`

HTTP	کد	زمان
400	<code>VALIDATION_ERROR</code>	خطای اعتبارسنجی ورودی یا قالب بدنه
401	<code>UNAUTHORIZED</code>	کلید/توکن ناقص، نامعتبر یا منقضی

HTTP	کد	زمان
403	FORBIDDEN	معتبر ولی بدون مجوز
404	NOT_FOUND	منبع پیدا نشد
501	NOT_IMPLEMENTED	مسیر محفوظ فاز بعد (بخش ۴.۹)
503	DEPENDENCY_UNAVAILABLE	سرویس SSO/فایل یا دیتابیس در دسترس نیست
500	INTERNAL_ERROR	خطای غیرمنتظره (جزئیات فقط در لاگ، نه برای کاربر)

جدول‌های دیتابیس مرتبط: ندارد.

وابستگی‌ها: ندارد (لایه API).

نکات و استثناءها: خطای ۵۰۰ هیچ‌وقت جزئیات داخلی را به کاربر نمی‌دهد؛ فقط در لاگ ثبت می‌شود.

۴.۹ مسیرهای محفوظ فاز دوم (در حال حاضر ۵۰۱)

این بخش چه کاری انجام می‌دهد؟

پنج مسیر «مدیریت درخواست» برای فازهای بعدی طراحی شده‌اند و اکنون آگاهانه پاسخ «پیاپی‌سازی نشده» (۵۰۱) می‌دهند تا مصرف‌کننده‌ها بدانند این قابلیت‌ها در راه‌اند.

چه کسی از آن استفاده می‌کند؟ اپراتور/کارتابل و شهروند (در آینده).

جزئیات عملکرد: مسیرها تعریف و مستند شده‌اند اما بدنه‌ی آن‌ها `NotImplementedException` پرتاب می‌کند.

مسیر	متد	کاربرد آینده
<code>/api/v1/requests/{id}</code>	GET	مشاهده‌ی جزئیات یک درخواست
<code>/api/v1/requests/by-tracking-code/{code}</code>	GET	پیگیری شهروند با کد پیگیری
<code>/api/v1/requests?status=&currentGroupId=&from=&to=</code>	GET	جستجو / کارتابل
<code>/api/v1/requests/{id}/status</code>	PUT	تغییر وضعیت
<code>/api/v1/requests/{id}/refer</code>	PUT	ارجاع به گروه دیگر

جدول‌های دیتابیس مرتبط: از جداول بخش ۴.۱ استفاده خواهند کرد (در فاز بعد).

وابستگی‌ها: ندارد (در انتظار فاز بعد).

نکات و استثناها: متدهای پشتیبان دیتابیس این عملیاتها (`SearchAsync` ، `ReferAsync` ، `UpdateStatusAsync`) در لایه `Infrastructure` نوشته شده‌اند اما هنوز به این مسیرها متصل نشده‌اند - در بخش ۷ علامت خورده است.

۴.۱۰ بررسی سلامت سرویس (Health Checks)

این بخش چه کاری انجام می‌دهد؟

دو نقطه‌ی ساده برای اطمینان از سالم بودن سرویس و دسترسی‌پذیری دیتابیس فراهم می‌کند؛ برای ناظرهای خودکار (systemd، Nginx، مانیتورینگ) استفاده می‌شود.

چه کسی از آن استفاده می‌کند؟ مدیر زیرساخت / سیستم‌های مانیتورینگ.

جزئیات عملکرد: `GET /health` و `GET /api/health` اجرا می‌شوند و اتصال به دیتابیس را با یک پرس‌وجوی ساده (`SELECT 1`) چک می‌کنند. پاسخ موفق: `Healthy`.

جدول‌های دیتابیس مرتبط: چک زنده‌بودن دیتابیس (بدون خواندن داده).

وابستگی‌ها: SQL Server.

نکات و استثناها: اگر دیتابیس در دسترس نباشد، پاسخ غیرموفق می‌شود و مانیتورینگ آن را ثبت می‌کند.

۴.۱۱ مستندات Swagger و احراز هویت API

این بخش چه کاری انجام می‌دهد؟

مستندات تعاملی (Swagger UI) API را در آدرس `/swagger` در اختیار توسعه‌دهندگان می‌گذارد تا هر مسیر، ورودی و خروجی را ببینند و تست کنند.

چه کسی از آن استفاده می‌کند؟ توسعه‌دهندگان و تست‌کننده‌ها.

جزئیات عملکرد:

- Swagger به‌صورت خودکار در حالت `Development` فعال است (این سرویس عمداً به‌صورت `Development` دیپلوی می‌شود تا Swagger همیشه باز باشد) و یا اگر `Swagger.Enabled=true` باشد.

- دکمه‌ی «Authorize» برای وارد کردن توکن Bearer و کلید API وجود دارد.

- خروجی JSON اعداد/تاریخ‌ها با `camelCase` و وضعیت‌ها (enum) به‌صورت متن ارسال می‌شوند؛ فیلدهای `null` در پاسخ حذف می‌شوند.

جدول‌های دیتابیس مرتبط: ندارد.

وابستگی‌ها: کتابخانه‌ی Swashbuckle.

نکات و استثناها: با اینکه در حالت `Development` هستیم، کلید واقعی دیتابیس تولید داخل تنظیمات `Development` آمده است - برای محیط واقعی باید `override` شود (بخش ۵).

۵. زیرساخت و تنظیمات

نحوه اجرای (توسعه)

```
dotnet run --project src/RequestService.Api # → http://127.0.0.1:5050 (Swagger در /swagger)
dotnet build RequestService.sln -c Release # ساخت خروجی نهایی
```

دیپلوی

سرویس عمده در حالت **Development** اجرا می‌شود (در Dockerfile، docker-compose و فایل systemd) تا **Swagger** همیشه در دسترس باشد. ابزارهای دیپلوی در رپازیتوری آماده‌اند:

فایل	کاربرد
Dockerfile	ساخت ایمیج چندمرحله‌ای لینوکسی، پورت ۵۰۵۰، محیط Development
docker-compose.yml	اجرای کانترینر (بدون Redis - سرویس بدون حال است)
deploy/requestservice.service	سرویس systemd برای آلمالینو (پورت 127.0.0.1:5050)
deploy/nginx.conf	پروکسی معکوس برای دامنه‌ی apiweb-137request.sabzevar.ir
deploy/deploy.sh	اسکرپت استقرار یک‌دست (publish + nginx + systemd)
postman/request-service.postman_collection.json	کالکشن Postman برای تست

تنظیمات مهم (نام کلید، بدون مقادیر حساس)

همه‌ی این‌ها از **appsettings.json** خوانده می‌شوند و می‌توانند با متغیرهای محیطی (به جای **:**) یا در **/systemd** Docker بازنویسی شوند:

کلید	توضیح	مقدار حساس؟
ConnectionStrings:Requests	رشته اتصال SQL Server (دیتابیس apiweb-137service)	بله (فقط در تنظیمات Development آمده است)

کلید	توضیح	مقدار حساس؟
<code>Sso:BaseUrl</code>	آدرس سرویس SSO (در توسعه <code>(http://127.0.0.1:5001)</code>)	خیر
<code>Sso:TimeoutSeconds</code> / <code>RetryCount</code> / <code>CircuitBreaker*</code>	مقاوم سازی فراخوانی SSO	خیر
<code>Files:BaseUrl</code>	آدرس سرویس فایل (در توسعه <code>(http://127.0.0.1:6000)</code>)	خیر
<code>Files:ValidationEnabled</code>	فعال/غیرفعال بودن تأیید فایل ها (در توسعه <code>(false)</code>)	خیر
<code>Files:ServiceToken</code>	توکن JWT سرویس به سرویس (توسط تیم سرویس فایل صادر می شود - هنوز خالی)	بله
<code>InternalAuth:HeaderName</code>	نام هدر کلید API (<code>X-API-Key</code>)	خیر
<code>InternalAuth:ApiKeys</code>	فهرست کلیدهای مجاز (در توسعه <code>(dev-internal-key-137)</code>)	بله
<code>Cors:AllowedOrigins</code>	دامنه های مجاز فرانت (پیش فرض <code>localhost</code> : <code>localhost:5173</code> و <code>3000</code>)	خیر
<code>Logging:File</code>	مسیر فایل لاگ سرور	خیر
<code>Swagger:Enabled</code>	روشن/خاموش کردن Swagger در غیر حالت توسعه	خیر

پورت ها و سلامت

- پورت سرویس: `5050`
- سلامت: `GET /health` و `GET /api/health`
- لاگ ها: کنسول + فایل گردشی روزانه (`/var/log/requestservice/app-.log` در سرور، `test-api-out.log` در توسعه)
- دامنه ی هدف: `apiweb-137request.sabzevar.ir`

امنیت

- سرویس پشت Nginx با HTTPS قرار می گیرد (گواهی Let's Encrypt).
- یونیت systemd با مجوزهای محدود (کاربر اختصاصی، `NoNewPrivileges`، `PrivateTmp`) اجرا می شود.
- هیچ لاگ/پیامی رمز واقعی را چاپ نمی کند.

• فایل‌های `bin` / `obj` و لاگ‌ها در `.gitignore` هستند.

۶. وابستگی به سرویس‌های خارجی / داخلی دیگر

سرویس	آدرس (توسعه)	چه کاری با آن انجام می‌شود	اگر در دسترس نباشد
-sso -login service	<code>http://127.0.0.1:5001</code>	اعتبارسنجی توکن و گرفتن کد ملی / تلفن (<code>GET /api/auth/me</code>)	خطای ۵۰۳ <code>DEPENDENCY_UNAVAILABLE</code> (درخواست شهروندی رد می‌شود)
files service	<code>http://127.0.0.1:6000</code>	تأیید وجود فایل و تشخیص نوع (<code>GET /api/files/{id}</code>) - فقط متادیتا	خطای ۵۰۳ (اگر <code>ValidationEnabled=true</code> ؛ در توسعه خاموش است)
SQL Server 2019	<code>185.255.91.242,2019</code>	نگهداری همه‌ی داده (<code>apiweb-137service</code>)	Health چک خراب و درج‌ها شکست می‌خورند

در فاز ۱ هیچ سرویس دیگری (پیامک، نقشه، پرداخت و ...) فراخوانی نمی‌شود.

۷. موارد ناتمام / نیاز به بررسی (TODO / Open Items)

- مسیرهای فاز دوم (۵۰۱) - مشاهده/پیگیری/جستجو/تغییر وضعیت/ارجاع هنوز پیاده‌سازی نشده‌اند. جالب است که متدهای لایه‌ی دیتابیس این عملیات‌ها (`GetByIdAsync` ، `SearchAsync` ، `ReferAsync` ، `UpdateStatusAsync`) قبلاً نوشته شده‌اند ولی به `Controller` متصل نشده‌اند - مشخص نیست این متدها آیا مطابق نیاز واقعی فاز بعد تنظیم شده‌اند یا نیاز به بازبینی دارند.
- توکن سرویس فایل (`Files:ServiceToken`) - هنوز صادر نشده؛ تأیید واقعی فایل فعال نیست. بلاک برای اتصال کامل به سرویس فایل.
- تناقض PII (اطلاعات شخصی) - مشخصات شخصی (نام/نام خانوادگی/تلفن) طبق درخواست در بدنه پذیرفته می‌شود ولی طبق تصمیم معماری نباید در `Requests` ذخیره شود؛ فعلاً فقط به صورت JSON در `RequestLogs.Description` ثبت می‌شود. نیاز به تأیید نهایی کارفرما (ادامه یا حذف این فیلدها).
- نبود مسیر ثبت/انتقال پروفایل در SSO - برای تکمیل پروفایل شهروند تلفنی، سرویس SSO هنوز مسیری برای دریافت/ثبت نام و تلفن ندارد.
- ناسازگاری کامنت و رفتار - در `RequestService.cs` ادعا شده «کد ملی مغایر در بدنه توسط اعتبارسنجی رد می‌شود» ولی چنین قانونی وجود ندارد (بدنه‌ی مغایر فقط نادیده گرفته می‌شود). باید کامنت یا کد اصلاح شود.
- کد مرده - متغیر `VideoExtensions` در `CreateRequestValidator` تعریف شده ولی استفاده نمی‌شود.

7. اعتبارسنجی فایل در توسعه – `Files:ValidationEnabled=false` در تنظیمات Development یعنی فایل‌ها تأیید

نمی‌شوند و نوع آن‌ها همیشه `Other` ذخیره می‌شود؛ این فقط برای توسعه است.

8. CORS نهایی – دامنه‌های نهایی فرانت هنوز در `Cors:AllowedOrigins` ثبت نشده‌اند (پیش‌فرض فقط localhost است).

9. اعمال Migration – ساختار دیتابیس (Migration `20260812061144_InitialCreate`) روی دیتابیس زنده اعمال شده است؛ برای محیط‌های جدید باید با `dotnet ef database update` اجرا شود.

10. تست با SSO/فایل واقعی – از محیط توسعه، این دو سرویس در دسترس نیستند؛ اعتبارسنجی واقعی باید روی سرور تست شود.

پایان مستند. این مستند بر اساس کد فعلی (فاز ۱) نوشته شده است؛ با پیاده‌سازی فازهای بعدی باید به‌روزرسانی شود.